

now anything remain in dashboard ? can we start giving application to the sales officer before doing the change in the application ui

Tuesday 3:54 PM

Yes, you can start giving the application to sales officers—but begin with a controlled pilot, not all 67 immediately.

Use the official link:

<https://ai.croplifescience.com/>

Recommended rollout:

1. Start with 5–10 sales officers from different states.
2. Give each officer their individual link/profile.
3. Ask for 2–3 genuine inspections daily.
4. Run the pilot for 3–5 days.
5. Review photo quality, diagnosis, territory mapping and contact information.
6. Complete the planned application UI improvements.
7. Then expand to all 67 officers.

The admin dashboard is ready for pilot monitoring, including:

- Sales-officer activity tracking
- State and territory filters
- Inspection and S3 image viewing
- Gemini, Gemma and Qwen comparisons
- Automatic training-data selection
- Qwen fine-tuning progress
- Model versions, metrics and quality gates
- Product catalogue approval and activation controls
- Employee and role management
- Architecture information
- Inspection deletion from database and S3

Remaining items are improvements rather than pilot blockers:

- Improve the farmer-facing mobile UI and translations.
- Collect enough real images for reliable Qwen validation.
- Test automatic GGUF/Ollama promotion only after a candidate passes the safety gates.
- Verify every sales officer's phone, email, state and territory.
- Push the latest commit to GitHub.
- Later replace temporary login/invite handling with Microsoft authentication.
- Add operational alerts for failed workers, low storage and unsuccessful training jobs.

Important: the first fine-tuned candidate was correctly rejected because it only had three validation cases and scored 0%. Therefore, sales officers will continue receiving results from the existing production AI flow—not from that weak experimental adapter.

So the best next step is: **start the small sales-officer pilot now while we improve the application UI in parallel.**

Tuesday 3:55 PM

refresh all the sales officer feild , so that it set again to send them fresh link and , as in testing we create links of some emplyee and full data through it

first change , make the crop life ai name to app name is clsl ai and a tag line for crop , so it market compamny also and its product also , as my sir told me structly to name app name as CLSL AI

second desgin the ui more goood , increse the pwa effect more , as it shiuld feel good ui effect and give feel like using mbile applicationonly

set the mascot proper

combine both the chabot together and make one common chatbot and in chatbot use the gemma api key only

remove unnesscary things and info from app , as you have add too much info , also with that you show somewher wrrtien in small gemini , product shwoing from clsl catelouge , cibrc, all this etc things removed

make cumposlry to select crop name , user must add crop name without it , it can not be move ahead, use all the crops name for the database must and you can add other crop name also , also add a serach option for seraching the crop name from the dropdwon and in last put option other(in other our model will recgnoze it)

also where user upload the photo they should show in small boxes that what they uoload

check that all things work in all lnagauge also which we add

do all this tasks complete , tell me when all this done, when it will be done we will check all changes and after that i gernate the link for the sales officer , they go on web app and from they downlaid the apk

Show more

Tuesday 4:27 PM

Context automatically compacted

I'll reset the sales-officer access/testing state so new links start clean, then update the app to **CLSL AI** with a cleaner mobile-first experience, one Gemma-powered assistant, mandatory

searchable crop selection, and multilingual checks. I'll preserve employee records and inspection history unless you specifically ask to remove them.

Ran commands

I've confirmed the existing field links are stored separately from employee records and inspection history, so they can be safely invalidated without deleting either. I'm adding a protected "reset all field links" control, then applying the app redesign and the crop-selection changes.

Today 10:25 AM

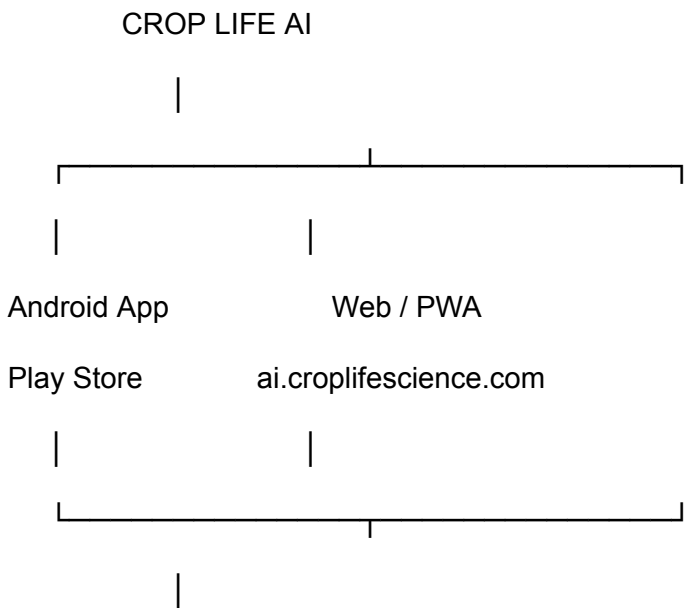
now the plan have a major changed , sir want to fully launch the app at once , sir is okay to use paid api key for crop inspection so now we have to desgin it the app and archeitueture that way and also upload on the playstore , so we need to make plan according to it

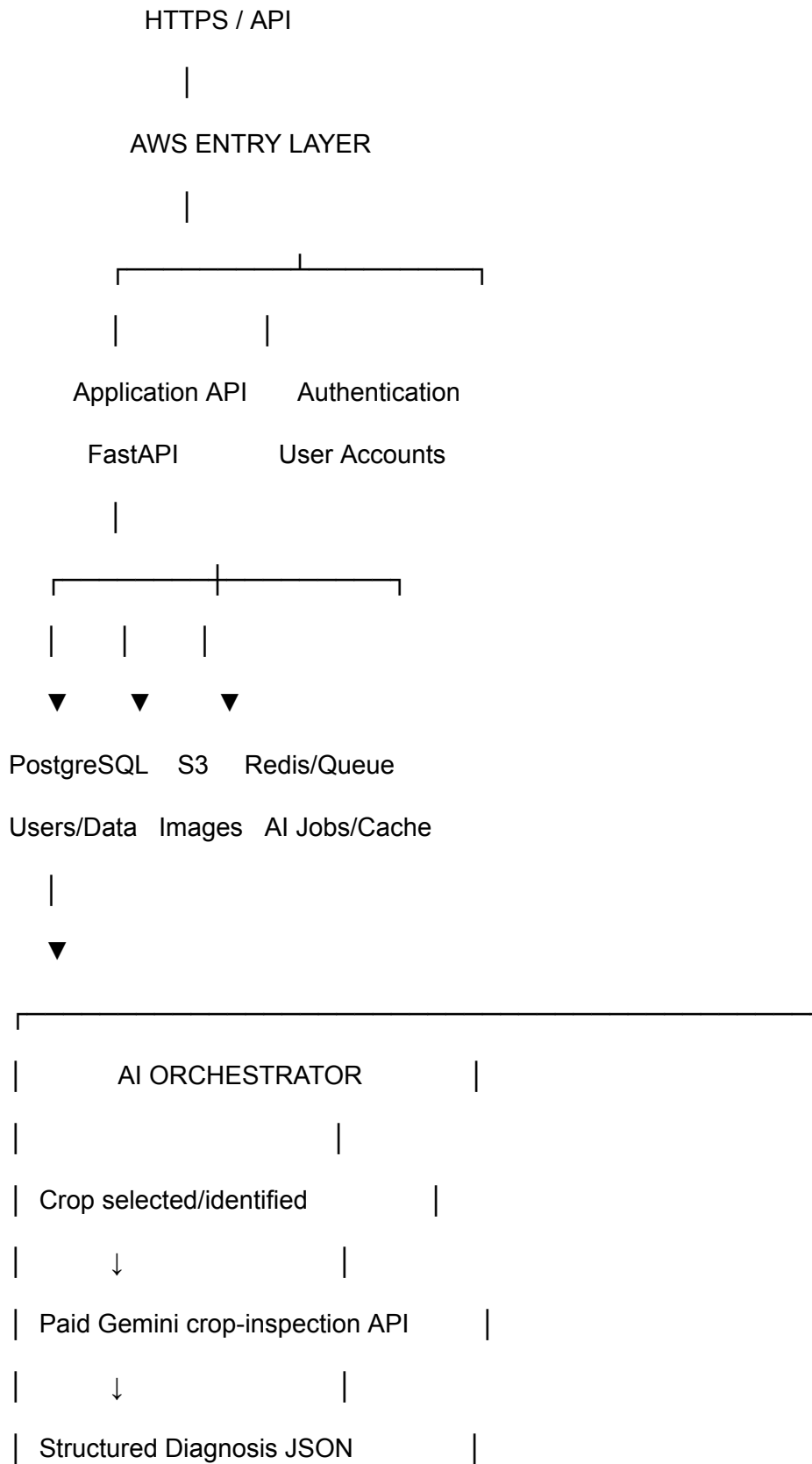
Yes. This changes the architecture significantly.

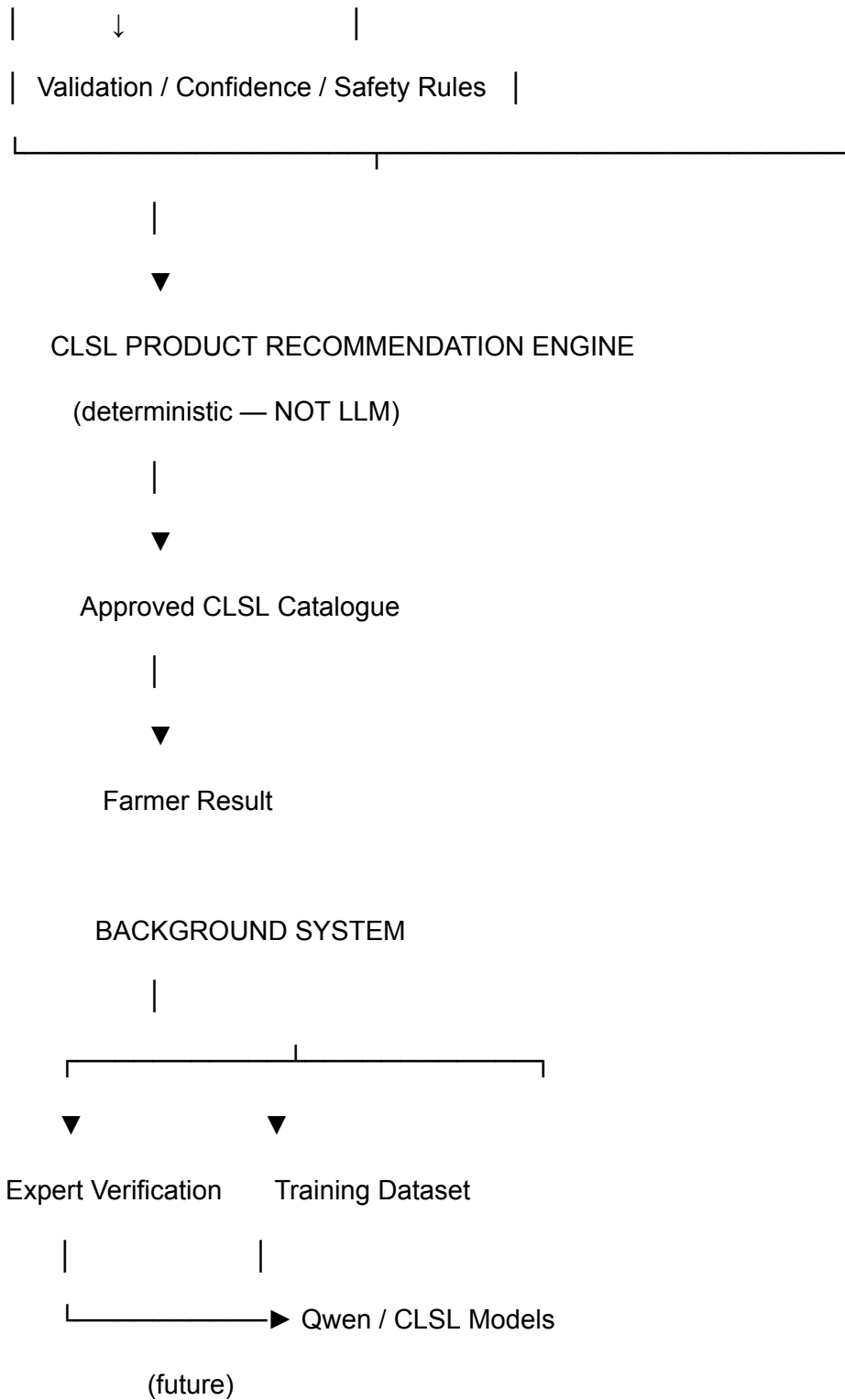
Previously, we were optimizing Crop Life AI around a controlled internal rollout, collecting training data, scheduled GPU/Qwen use, and gradually replacing Gemini. If management now wants a **full public launch + paid API + Play Store**, we should design it as a production consumer application from the beginning rather than scaling the current employee pilot architecture piece by piece.

Revised Crop Life AI launch architecture

I would structure the production system like this:







The critical rule remains unchanged: **Gemini diagnoses; it does not select Crop Life products.** Your approved catalogue engine remains responsible for recommendations.

What changes now

The biggest change is that I would **not block launch on Qwen training**.

For launch:

Paid Gemini API → primary production crop diagnosis

Your Qwen work becomes a parallel R&D/training pipeline:

Public users

↓

Gemini production diagnosis

↓

Expert verification + quality filtering

↓

High-quality CLSL dataset

↓

Qwen training

↓

Shadow evaluation

↓

Eventually partial/primary production

This gets the product launched while simultaneously creating the dataset needed to reduce Gemini dependency later.

And because it is now public, I would stop planning around the GPU being available only around 8–10:30 PM as part of the **live inference path**. The GPU can still run scheduled training jobs, but a public farmer should never have to wait for it.

Product scope for V1 public launch

We should now freeze a proper launch scope rather than continuing to add features randomly.

Farmer-facing Android + Web

I recommend the launch app contain:

Crop Inspection

Take Photo / Upload Photos



Choose Crop OR AI-assisted crop detection



Symptoms / optional description



Location (with permission)



Analyze



Crop

Probable problem

Disease / pest / deficiency

Confidence

Visible symptoms

Explanation

Immediate actions

Prevention



Relevant approved CLSL products

↓

Dose / application / catalogue information

For public farmers, unlike the controlled Sales Officer dataset flow, we can allow **AI-assisted crop identification** for convenience. But internally keep `user_selected_crop`, `ai_predicted_crop`, and expert-verified crop as separate fields so public data doesn't corrupt training labels.

Crop Life Mitra / AI Assistant should answer crop protection and farming questions, support local languages, understand the inspection result, and use CLSL catalogue/RAG where company-product information is involved.

Then include the **CLSL product catalogue**, crop/pest/disease library, calculators, weather/location-based information where useful, inspection history, saved results, profile, language selection, help/support, privacy/account controls, and feedback/report-wrong-result capability.

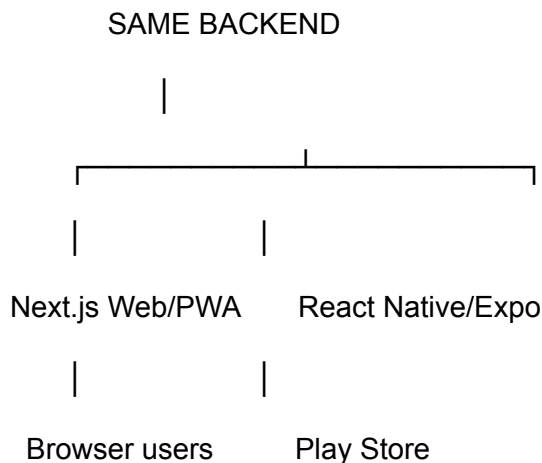
Android: don't simply wrap the website

Since management specifically wants Play Store distribution, I would build:

React Native + Expo

for Android while retaining the existing web/PWA.

We do not build another backend.



Both applications use the same:

FastAPI

PostgreSQL

S3

Gemini

CLSL product engine

Authentication

Admin system

Expert review system

That is much easier to maintain than two independent systems.

AWS needs to become production-grade

Your current single `m7i-flex.large` EC2 can remain useful during development, but I would **not architect the public product around one EC2 forever**.

Target architecture:

Route 53 / existing DNS



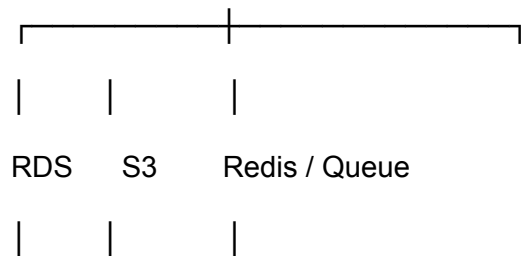
CloudFront / WAF



Application Load Balancer



App/API compute



RDS S3 Redis / Queue

DB Images background work

Eventually:

RDS PostgreSQL instead of production PostgreSQL living inside the same EC2 server as the app.

S3 for crop images.

CloudFront for static/image delivery where appropriate.

AWS WAF for public API protection.

Secrets Manager/SSM for production secrets.

CloudWatch for logs, metrics and alerts.

ALB + scalable compute once traffic warrants it.

We don't need to buy all of this today. We can migrate in stages before launch so you don't generate unnecessary AWS bills during development.

Paid Gemini also changes our API design

Never put the paid Gemini key inside the Android APK.

This would be dangerous:

Android → Gemini ❌

The correct flow is:

Android

↓

Crop Life API

↓

authentication

rate limiting

image validation

usage limits

abuse protection



Gemini API

The API key exists **server-side only**.

That becomes especially important once the APK is on Play Store because anyone can inspect an APK.

We should also implement per-user/device/IP limits and company-wide daily spending controls so an abusive client cannot create a huge Gemini bill.

Database needs expansion

Our current employee-oriented database needs to evolve into a public platform database.

Keep existing:

employees

roles

products

product_categories

crop/problem mappings

approvals

audit records

Add production concepts such as:

users

user_profiles

devices

farms

fields

inspections

inspection_images

ai_diagnoses

diagnosis_versions

expert_reviews

feedback

saved_products

chat_sessions

chat_messages

usage_events

api_usage

notifications

consents

model_runs

model_metrics

And importantly:

model_provider

model_name

input_tokens

output_tokens

latency_ms

estimated_cost

confidence

fallback_used

Then your Admin Dashboard can show management things like:

Users

Daily active users

Inspections today

Successful diagnosis %

Average response time

Gemini calls

Gemini cost

Top crops

Top diseases

Locations

CLSL products recommended

Expert disagreement rate

Qwen vs Gemini accuracy

Dataset growth

That will become very valuable for CLSL.

Play Store adds another workstream

The Play Store launch isn't just generating an APK.

We will need:

Crop Life AI Android application

Production package/application ID

App signing

AAB build

App icon

Splash screen

Screenshots

Feature graphic

App description

Privacy Policy

Terms / disclaimer

Data safety declaration

Permissions justification

Camera permission

Photo/media handling

Location permission (if used)

Account deletion/data handling where applicable

Internal testing

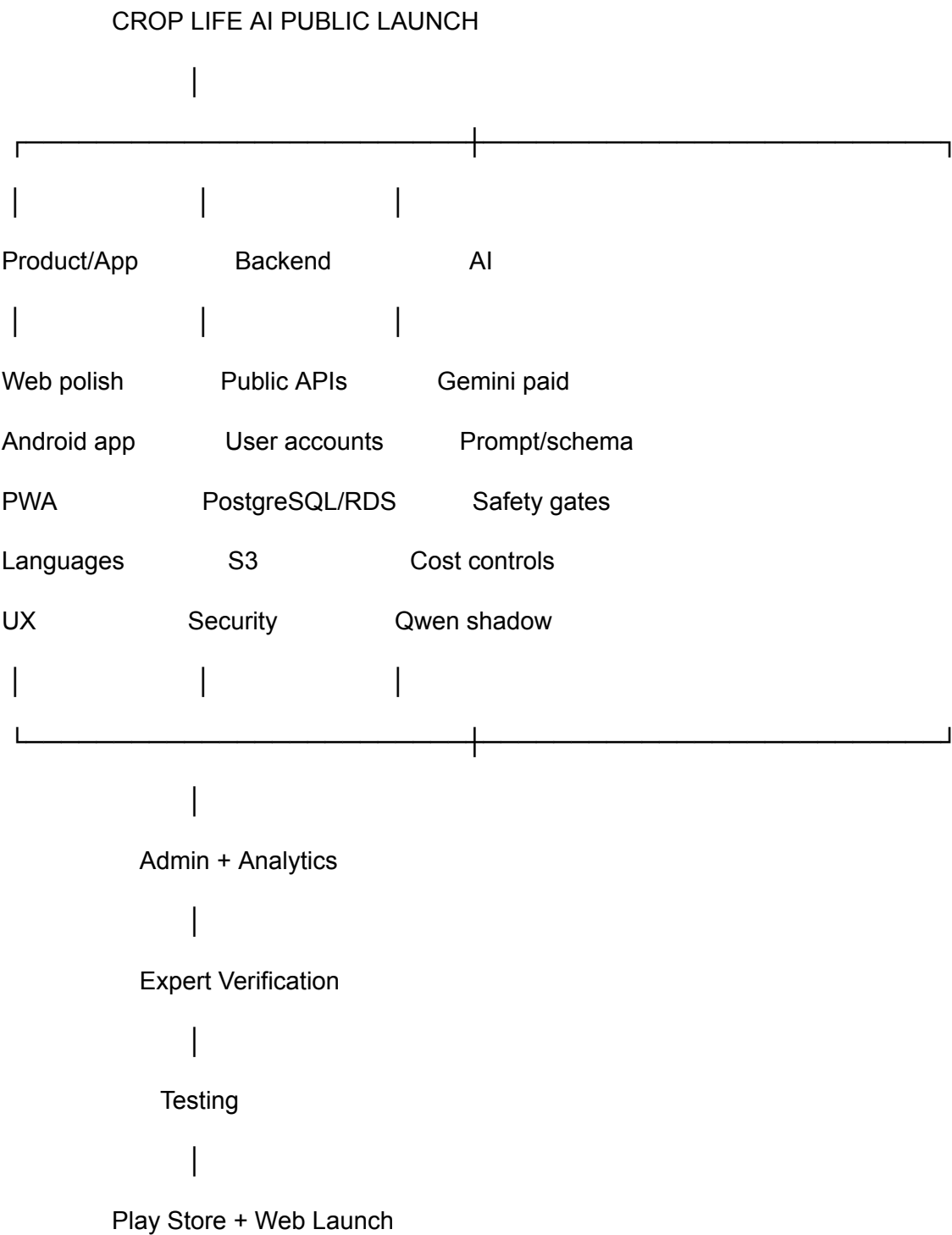
Closed testing if required

Production release

Because this application deals with crop diagnosis and pesticide/product recommendations, we should also design the wording and safety layer carefully: probable diagnosis/confidence, label-compliant use information, and appropriate guidance to consult the product label/local agronomist where needed rather than presenting uncertain AI output as guaranteed fact.

Revised execution plan

Instead of a six-month research-first roadmap, I'd reorganize the project into one **launch program** with parallel workstreams:



Phase 1 — freeze architecture and requirements

Do this **now**, before more coding.

Define every launch screen, every backend service, authentication, public vs employee flows, database schema, Gemini model strategy, product recommendation rules, admin capabilities, languages, permissions, analytics and Play Store requirements.

Phase 2 — production backend

Upgrade database/schema, S3 image pipeline, user authentication, inspection APIs, Gemini gateway, rate limiting, cost tracking, catalogue engine, expert review, audit logs and admin APIs.

Phase 3 — finish web + build Android

Keep the existing web application and build React Native/Expo against the same APIs.

Phase 4 — AI production validation

Test hundreds of representative cases before public release. Measure diagnosis quality rather than simply whether the API returns successfully.

Phase 5 — infrastructure hardening

RDS migration, backups, monitoring, secrets, rate limits, WAF, logging, recovery testing and scaling.

Phase 6 — launch validation

Android internal testing → production candidate → Play Store submission, while web production runs at:

ai.croplifescience.com

Then controlled rollout followed by broader promotion.

One change I would make immediately

Pause the QLoRA activation work as the critical path.

Don't delete the GPU or Qwen work. Keep it as the future-model track. But right now your engineering priority should become:

1. Production architecture
2. Paid Gemini integration
3. Public-user system
4. Database + S3
5. Production security/cost controls
6. Android application
7. Admin/expert system
8. End-to-end testing
9. Play Store
10. Qwen improvement in parallel

This gives management a launchable product without throwing away the work you've already done.

Before we tell Codex to make more changes, I recommend we create a **single Crop Life AI Public Launch Master Plan** containing the final architecture, exact app screens, database design, AWS components, Gemini architecture, Android architecture, admin dashboard, security, testing, deployment and Play Store checklist. That becomes the blueprint we follow rather than changing architecture every few days.

For **crop photo inspection/disease diagnosis**, between the two you asked about, I would start production testing with **Gemini 3.5 Flash**, not 2.5 Flash. It is GA/stable and Google describes it as the newer, more capable Flash generation; both accept multimodal inputs.

But there is an important update: as of September 2026, Google also has **Gemini 3.6, 3.7 and 3.8 Flash**. So before we lock the Play Store production model, I would benchmark 2.5 Flash vs 3.5 Flash and at least the current 3.x Flash option on our actual crop cases. Google's release notes show these newer models are already GA.

2.5 Flash vs 3.5 Flash

	Gemini 2.5 Flash	Gemini 3.5 Flash
Image input		

Reasoning	✓	✓ stronger/newer generation
Production	GA	GA
Input	\$0.30 / 1M tokens	\$1.50 / 1M tokens
Output + thinking	\$2.50 / 1M	\$9.00 / 1M
My use for CLSL	cheaper benchmark/fallback	primary candidate

These are Google's current Standard paid API rates.

What will one crop inspection cost?

There isn't a fixed "price per request." Google bills according to the tokens consumed by the images, prompt, output, and thinking.

For Crop Life AI, suppose one inspection is:

4 crop photos

+ crop/context information

+ our diagnosis prompt

+ structured JSON result

+ reasoning/thinking

For illustration, if a request ended up using **~5,000 input tokens + ~2,000 billed output/thinking tokens**, then:

Gemini 3.5 Flash

Input:

$5,000 \times \$1.50 / 1,000,000$

= \$0.0075

Output/thinking:

$2,000 \times \$9 / 1,000,000$

= \$0.018

Total $\approx$ \$0.0255

At roughly ₹83–₹85/\$ for simple budgeting, that's about **₹2.1 per inspection**.

If we optimize image resolution, prompt length and thinking/output, it could be substantially lower; if thinking or image/token consumption is much higher, it can be higher. So I would initially budget roughly **₹1–₹3 per 4-photo inspection** until we measure actual production token usage.

At 2,000 inspections/month, that planning range is roughly:

$₹1 \times 2,000 = ₹2,000/\text{month}$

$₹2 \times 2,000 = ₹4,000/\text{month}$

$₹3 \times 2,000 = ₹6,000/\text{month}$

That's **Gemini inspection cost only**, not AWS/S3/database/etc.

One thing I want us to do before choosing

Don't select a model because Google says one is newer.

Take **30–50 real crop cases**, preferably with agronomist-confirmed answers, and send exactly the same four images and prompt to each candidate. Compare disease/pest identification, false diagnoses, confidence calibration, JSON reliability, latency, and actual cost.

For your app, **diagnostic accuracy matters much more than saving ₹1 on a request**.

So my proposed production design is:

4 crop photos

↓

Gemini 3.x Flash primary

↓

Structured diagnosis JSON

↓

Confidence / validation

↓

CLSL deterministic product engine

↓

Approved CLSL products

And **Gemini still never chooses the CLSL product.**

If you want, next I can calculate the cost much more accurately for **100, 500, 1,000, 5,000 and 10,000 crop inspections/day**, and then we can decide which Gemini model makes financial sense for the full public launch.

tell me in inr , dollar is 97 rs here for sales officer only 4 photos cumposlry , for other people it will be they can upload any number of image

Yes. Using **\$1 = ₹97**, and separating your two flows makes sense.

One important point first: for public users I would allow a flexible number of photos in the UI, but **still set a maximum**—for example 1–5 photos per inspection. Otherwise someone could upload 20–50 images and unnecessarily increase API cost and processing time.

Using the Gemini 3.5 Flash pricing we discussed—**\$1.50/1M input tokens and \$9/1M output/thinking tokens**—the INR rates are approximately **₹145.50 per 1M input tokens** and **₹873 per 1M output/thinking tokens**.

Practical Crop Life AI budget

For planning, I would use these ranges rather than assuming every request costs exactly the same:

Inspection

Photos

Estimated Gemini
cost/request

Public basic	1	~₹0.7–₹1.5
Public	2	~₹0.9–₹1.8
Public	3	~₹1.1–₹2.2
Sales Officer	4	~₹1.3–₹2.7
	compulsory	
Public maximum	5	~₹1.5–₹3.0

The variation comes largely from image size, reasoning/thinking tokens and response length—not just the number of photos.

For budgeting, I'd therefore assume **₹2 per Sales Officer inspection** initially, with **₹3 as a conservative ceiling/budget allowance** until we have real usage data.

Your 67 Sales Officers doing 2–3 inspections/day would mean roughly **134–201 inspections/day**, or around **2,680–4,020 inspections over 20 working days**. At ₹2/request, that's approximately **₹5,360–₹8,040/month**; using the safer ₹3/request budget gives **₹8,040–₹12,060/month**.

For the public app

I'd implement:

Sales Officer

↓

Exactly/minimum 4 required photos

↓

Gemini analysis

Public farmer

↓

1–5 photos

↓

Gemini analysis

But there's an optimization I'd strongly recommend: **don't send original 5–15 MB phone photos directly to Gemini**. Preserve the useful original in S3 if we want it for the CLSL training dataset, but generate an optimized analysis copy and send that to Gemini.

That gives us:

Original photo

└──→ S3 → future CLSL/Qwen training

|

└──→ optimized image → Gemini → diagnosis

So we preserve valuable high-quality training data while controlling Gemini latency/cost.

And once the production version is running, we'll record the **actual Gemini token usage and ₹ cost for every inspection** in the database. Then the admin dashboard can show something like:

Today's inspections 186

Gemini requests 186

Average cost/inspection ₹1.74

Today's AI cost ₹323.64

Monthly AI cost ₹7,846

Average response time 6.8 sec

That is much better than relying permanently on estimates.

For the public launch, I would currently design around **Gemini 3.5 Flash as the primary candidate, 1–5 photos for public users, and compulsory 4-photo collection for Sales Officers**, while we benchmark it against the other Gemini models before locking the production model.

The new direction is:

Public Web / Android App



CLSL AI Backend



Paid Gemini diagnosis API



Safety + confidence rules



Approved CLSL product engine



Farmer result

Qwen remains a background R&D/training model. It must not delay or power the live public diagnosis until it proves better through evaluation.

What we keep from current work:

- CLSL product catalogue, approved crop mapping and product controls
- PostgreSQL, S3 image storage, admin portal, employee/sales system
- PWA/web app, languages, weather, calculators and Crop Life Mitra
- Gemini/Gemma/Qwen comparison and training pipeline

What changes for public launch:

1. Paid Gemini becomes the reliable live image-diagnosis provider.
2. Public users get accounts, profiles, inspection history, feedback and privacy controls.
3. API keys remain only on AWS — never inside the Android app.
4. Add rate limits, usage tracking, spending limits and abuse protection.
5. Separate public farmer data from Sales Officer data.
6. Build one shared API for both web/PWA and Android.
7. Create Android app with React Native + Expo, then publish an AAB to Play Store.
8. Upgrade AWS security, monitoring, backups, secrets and eventually move production database to RDS.
9. Add public legal pages: Privacy Policy, Terms, AI diagnosis disclaimer, data deletion and Play Store data-safety disclosures.

The immediate priority order should be:

1. Final public-launch blueprint
2. Paid Gemini gateway and cost controls
3. Public user database/authentication
4. Public inspection, history, feedback and support APIs
5. Web/PWA completion as **CLSL AI**
6. Android application
7. Production testing and security hardening
8. Play Store testing and submission
9. Qwen improvement in parallel

Before I change the backend architecture, I need these three launch decisions:

- Which Google Cloud project will hold the paid Gemini billing/API key?
- Do you already have a Google Play Console developer account under CLSL?
- For farmers, should sign-in be mobile OTP, Google sign-in, or both?

Once you confirm these, I will create the single CLSL AI Public Launch Master Plan and begin restructuring the production system around it.

10:36 AM

for sales officer only 4 photos compsostry , for other people it will be they can upload any number of image

i will create a new CLSL_AI project from my accoubnt and will use gemini 3.5 flash
no we dont have Google Play Console developer account under CLSL
sign-in be mobile OTP, Google sign-in, or both
also sir want a normal login for all people and farmers and for our company dealer, like they have some qr code they give the to the farmer to famrer and farmer login thriugh it so we go the info of that this farmer is associate with this dealrer , our main reason begeind all this is to collect the farmer data or after downloding there will be refer code scan to this , we want the data of farmers , so also dealer will tell downlaing and login thriugh this app give you some rs disciunt coupun inside it and that farmers can use and get dsicunt form the farmers and dealer give that much discunt to famrer, our mmain goal is to make the famers to touch with us directly as we have a huge dealers betwwen us , we are not drielctly serving to famrer, so thats why this app is insitate , to cnnect and collect famrer data

10:51 AM

Worked for 20s

This is the right direction. CLSL AI should become a direct farmer-connect platform, while still respecting the dealer network.

I have updated the launch design accordingly:

User type	Photos	Login	Purpose
-----------	--------	-------	---------

Sales Officer	Minimum 4	Personal CLSL field link	Structured field data and model-quality images
Farmer / public user	1–5 photos	Mobile number + OTP	Crop support, history and direct CLSL relationship
Dealer	Company login	Dealer dashboard	QR/referral codes, farmer attribution and coupons
CLSL admin	Secure company login	Admin portal	Catalogue, dealer controls, farmer data, campaigns and analytics

For public farmers, crop selection should be optional: they can select the crop, or choose “Other” and let the AI identify it. Sales Officers will still have the mandatory crop selection and four-photo flow.

The correct referral journey is:

Dealer QR / referral code



Farmer installs CLSL AI



Mobile OTP login + consent



Referral is saved permanently to farmer profile



Farmer uses crop inspection and CLSL AI



Eligible coupon appears in the app



Dealer validates coupon during purchase



CLSL sees farmer, dealer, location, crop and engagement data

The QR code must contain a signed referral token, not simply a visible dealer ID. This prevents farmers from changing or guessing dealer codes.

For login, I recommend:

- Mobile number + OTP as the main farmer sign-in.
- Optional Google sign-in later for convenience.
- Email/password or Microsoft login for CLSL employees, dealers and administrators.
- No login before a farmer can explore basic app information; login becomes compulsory before inspection, saved history, coupons or personal recommendations.

Your paid primary diagnosis model should be [gemini-3.5-flash](#). It supports image input and structured outputs, and Google lists it as a stable model. [Gemini 3.5 Flash documentation](#)

The production model roles will be:

Farmer crop inspection → Paid Gemini 3.5 Flash

Farmer/common chatbot → Paid Gemini 3.5 Flash

CLSL product selection → Deterministic CLSL catalogue engine

Qwen → Shadow evaluation and future fine-tuning only

The next public-launch blueprint will add these data areas:

- Farmer accounts, OTP verification and consent
- Dealer accounts, territories, QR codes and referral campaigns
- Referral attribution history
- Coupons, eligibility, redemption and abuse checks
- Farmer inspection history and feedback

- Gemini usage, latency and cost tracking
- Dealer/farmer analytics in the admin portal
- Public privacy, terms, data-deletion and support flows

You do not yet need a Play Console account to build the Android app, but CLSL will need one before releasing publicly on Play Store. We will create the Android app as a separate React Native/Expo client using the same secure backend as the web app.

The immediate next work should be the Public Launch Master Plan, followed by public authentication and the dealer QR/coupon system. I will keep the current CLSL AI interface work, but shift it toward this farmer-and-dealer launch model rather than only the Sales Officer pilot.

10:52 AM

so decide the plan in detail and we dont want to make a app using andorioed studio , we discussed that we will load the web app in the apk , like the amazon does , if you remembereed we discussed thtis thing

10:55 AM

Yes, I remember: we will keep one CLSL AI web/PWA codebase and package it for Play Store, not build a separate React Native or Android Studio application. I'm confirming the safest Play Store packaging approach so the plan uses the web app directly without exposing API keys.

Searched the web for trusted web activity PWA Play Store Bubblewrap Digital Asset Links | developer.android.com ...

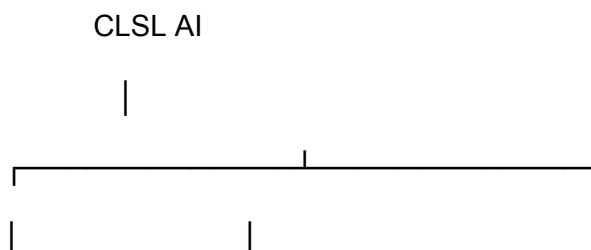
Context automatically compacted

You've hit your usage limit. Upgrade your plan or add credits to continue, or try again later.

Yes — with the new management direction, I would change one part of the previous plan:

We do not need React Native or a separately developed Android app. We keep **one CLSL AI web/PWA codebase** and package that web experience for Google Play. Your existing ai.croplifescience.com remains the real application and AWS remains the backend.

Final CLSL AI public-launch architecture

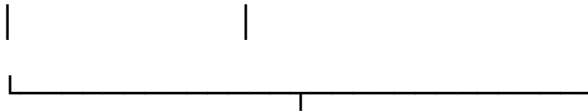


ai.croplifescience.com

Google Play

Web / PWA

Android package

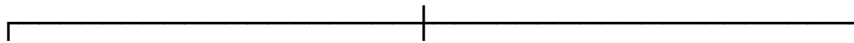


SAME WEB APPLICATION



CLSL AI Backend

AWS



PostgreSQL

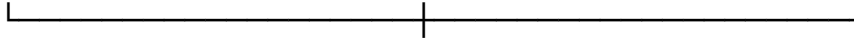
S3

Authentication



Crop images

OTP/users



AI Diagnosis Gateway



Gemini 3.5 Flash PAID



Diagnosis JSON only



Safety/confidence checks



CLSL PRODUCT ENGINE

(NO LLM selection)



Approved CLSL catalogue



Farmer result

For the Play Store packaging, the appropriate architecture to investigate is Android's **Trusted Web Activity (TWA)** approach for a PWA/web experience, with domain/app verification through Digital Asset Links, rather than creating a second React Native product. We should validate the exact current Play requirements when we reach packaging.

That gives us **one application to maintain**, which is exactly what you want.

1. User system

Now I'd divide users into four major groups.

User	Login	Inspection photos	Purpose
Farmer/Public	Mobile OTP	1–5 recommended	Diagnosis + direct CLSL relationship

Sales Officer	CLSL identity	4 compulsory minimum	High-quality structured dataset
Dealer	Dealer credentials/OTP	As needed	Farmer/referral/coupon network
CLSL Admin/Expert	Microsoft/company auth	—	Management/review

I would **not literally allow unlimited images** from public users. Let them choose how many are useful, but cap an inspection at something like **5 images**.

Otherwise one person could upload 30 images and consume unnecessary Gemini, bandwidth and S3 resources.

Sales Officers remain special:

Select crop



Photo 1 — whole plant

Photo 2 — affected part

Photo 3 — close symptom

Photo 4 — different angle/underside



Submit enabled

That data is much more valuable for future CLSL model training.

2. Farmer acquisition is now a core product

What your sir wants isn't merely a crop-diagnosis application.

The business objective is:

Current situation

CLSL



Distributor



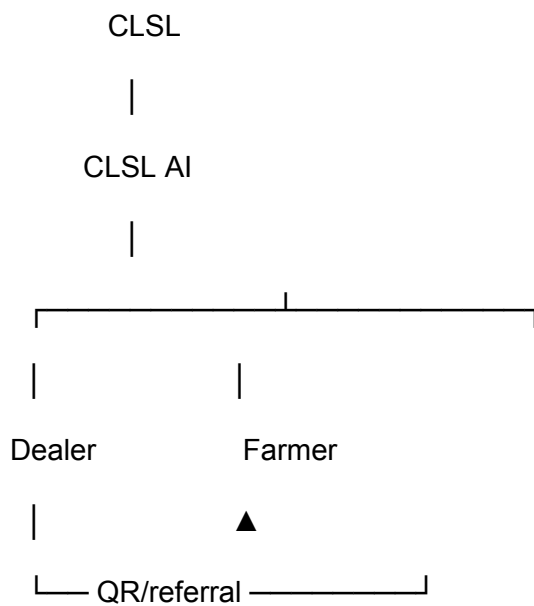
Dealer



Farmer

CLSL doesn't directly know enough about farmer

CLSL AI creates another relationship:



The dealer doesn't disappear. Instead, the dealer becomes an **acquisition channel**.

That is a very important distinction for the architecture.

3. Dealer QR/referral system

Each approved dealer gets their own QR/referral mechanism.

For example:

Dealer: ABC Agro Center

[QR CODE]

Referral:

CLSL-A8X92K

Farmer sees a poster/card at the shop:

Download CLSL AI
Scan this QR / enter referral code
Register and receive your CLSL welcome coupon.

Flow:

Dealer QR

↓

ai.croplifescience.com/join/<signed-token>

↓

PWA / Android package opens

↓

Farmer mobile OTP

↓

Consent



Farmer account created



dealer_id recorded



Welcome coupon generated



Farmer uses CLSL AI

We should retain both:

acquisition_dealer_id

current/preferred_dealer_id

rather than overwriting the original acquisition attribution later.

4. Coupon system

This needs to be designed properly rather than simply displaying a discount number.

Example:

WELCOME TO CLSL AI

₹50 OFF

Coupon:

CLSL-8K7P2X

Valid at:

ABC Agro Center

Expires:

30 Sep 2026

At purchase:

Farmer shows coupon



Dealer dashboard



Dealer enters/scans coupon



Backend validates



Valid / unused / not expired



Dealer gives discount



Redeemed

The database records:

Farmer

Dealer

Coupon

Campaign

Amount

Issue date

Expiry

Redemption date

Purchase reference if applicable

Status

This becomes measurable marketing rather than uncontrolled discounting.

And we should define with management **who economically funds/reimburses the discount and the settlement process** before public launch.

5. Farmer data

Since farmer relationship/data is a major business objective, we should collect **useful data transparently and with consent**, not simply collect everything possible.

The farmer profile can gradually build:

Mobile number

Name

Preferred language

State

District

Village / approximate location

Dealer/referral source

Crops grown

Farm/acreage information (optional)

Inspection history

Disease/pest history

Products viewed

Coupons issued/redeemed

Feedback

App engagement

Then CLSL eventually gets aggregate insights such as:

42,580 registered farmers

Gujarat	18,240
---------	--------

Maharashtra	9,510
-------------	-------

Rajasthan	6,820
-----------	-------

...

Cotton farmers	12,340
----------------	--------

Tomato	7,810
--------	-------

Chilli	6,210
--------	-------

Whitefly reports	826
------------------	-----

Early blight	643
--------------	-----

Thrips	581
--------	-----

...

Dealer ABC

Farmers acquired 386

Active farmers 247

Coupons redeemed 92

That could become one of the highest-value parts of the system.

6. Login decision

For farmers, my recommended launch authentication is:

Mobile number + OTP.

Don't make Google login compulsory.

Your target includes farmers who may not remember Google credentials but almost certainly know their phone number and can receive an OTP.

We can later add:

Continue with mobile number ← primary

Continue with Google ← optional

For dealers, we can use dealer mobile OTP/accounts.

For CLSL employees/admins, keep the stronger company authentication system.

7. Gemini

You already made the model decision:

gemini-3.5-flash paid tier.

Google currently lists paid Standard pricing as \$1.50/1M input tokens and \$9/1M output tokens including thinking.

Create the new Google project as something clearly production-specific, for example:

CLSL-AI-Production

But don't put the API key into JavaScript accessible to the browser/PWA or into the Android package.

Always:

Farmer



CLSL AWS API



Gemini 3.5 Flash

Never:

Farmer browser/APK



Gemini directly ❌

And our permanent product rule stays:

Gemini



Disease/pest/deficiency diagnosis

NOT

Gemini



Choose pesticide ❌

Product selection remains CLSL's controlled catalogue engine.

8. Qwen changes priority

We don't throw Qwen away.

But it moves out of the public request path:

LIVE

Farmer → Gemini 3.5 Flash → result

Meanwhile:

BACKGROUND

Verified inspections



Training dataset



Qwen QLoRA



Evaluation



Compare against Gemini



Only promote when sufficiently validated

Therefore the GPU training work can continue, but **it cannot hold up launch.**

9. Play Store

Because this is a CLSL commercial product, I recommend that the Play developer presence be created as an **organization/business account**, not under your personal developer identity.

Google says organization developer accounts require organization details including a **D-U-N-S number**, organization address/site and verified contact information; verification may also require an authorized representative's identity and organization documentation. Google warns that obtaining a D-U-N-S number can take time, so this should be started early.

So management/HR/accounts should start checking whether **Crop Life Science Limited already has a D-U-N-S number**.

That's one task I wouldn't leave until the end.

10. Revised development sequence

Now I would lock the work into this sequence:

PHASE 1 — MASTER DESIGN

↓

Farmer/dealer/referral/coupon architecture

Database schema

Permissions

Privacy/consent

Gemini gateway

↓

PHASE 2 — BACKEND

↓

Farmer accounts + OTP

Dealer accounts

QR/referrals

Coupons

Public inspections

History

Feedback

Gemini cost/rate limits



PHASE 3 — WEB/PWA



Farmer onboarding

Dealer referral journey

Inspection

Mitra

Catalogue

Weather

Calculators

History

Coupons

Profile



PHASE 4 — ADMIN



Farmer analytics

Dealer analytics

Coupon campaigns

AI usage/cost

Inspection analytics

Expert verification



PHASE 5 — PRODUCTION HARDENING



S3

DB backups / RDS migration

Monitoring

Rate limiting

Secrets

Abuse controls

Security testing



PHASE 6 — ANDROID PACKAGING



PWA

TWA/package

Digital Asset Links

AAB

Play Console



PHASE 7 — TEST



Internal users

Dealers

Farmers

AI validation

Coupon validation

Load/security testing



PHASE 8 — PUBLIC LAUNCH

One useful Play Store detail: Google requires app metadata, privacy/data-safety information, and where reviewers need authenticated access, review credentials/resources such as login information or a QR code.

What I would do right now

Don't give Codex another isolated feature request yet.

We've reached the point where farmer acquisition, dealers, coupons, OTP, paid Gemini, PWA/APK, analytics and privacy all affect the database and APIs.

The next artifact should be the **CLSL AI Public Launch Master Plan v1**, including the complete screen map, database tables/relationships, API endpoints, AWS architecture, farmer/dealer journeys, coupon rules, Gemini gateway, admin dashboards, PWA/TWA Play Store architecture, security, privacy, testing and phased implementation.

Once that blueprint is frozen, we can give Codex implementation tasks **phase by phase without repeatedly**

Edited files, ran commands

You've hit your usage limit. Upgrade your plan or add credits to continue, or try again at Sep 19, 2026, 4:03 PM.

Today 10:36 AM

now the plan have a major changed , sir want to fully launch the app at once ,...

A major changed, sir dont want to make wait for 6 months so sir said use the api key we will pay the cost, so we need to make the plan according to it

10:36 AM

Worked for 27s

Yes — we should now change to a public-launch architecture.

The new direction is:

Public Web / Android App



CLSL AI Backend



Paid Gemini diagnosis API



Safety + confidence rules



Approved CLSL product engine



Farmer result

Qwen remains a background R&D/training model. It must not delay or power the live public diagnosis until it proves better through evaluation.

What we keep from current work:

- CLSL product catalogue, approved crop mapping and product controls
- PostgreSQL, S3 image storage, admin portal, employee/sales system
- PWA/web app, languages, weather, calculators and Crop Life Mitra
- Gemini/Gemma/Qwen comparison and training pipeline

What changes for public launch:

1. Paid Gemini becomes the reliable live image-diagnosis provider.
2. Public users get accounts, profiles, inspection history, feedback and privacy controls.
3. API keys remain only on AWS — never inside the Android app.
4. Add rate limits, usage tracking, spending limits and abuse protection.
5. Separate public farmer data from Sales Officer data.
6. Build one shared API for both web/PWA and Android.
7. Create Android app with React Native + Expo, then publish an AAB to Play Store.
8. Upgrade AWS security, monitoring, backups, secrets and eventually move production database to RDS.
9. Add public legal pages: Privacy Policy, Terms, AI diagnosis disclaimer, data deletion and Play Store data-safety disclosures.

The immediate priority order should be:

1. Final public-launch blueprint

2. Paid Gemini gateway and cost controls
3. Public user database/authentication
4. Public inspection, history, feedback and support APIs
5. Web/PWA completion as **CLSL AI**
6. Android application
7. Production testing and security hardening
8. Play Store testing and submission
9. Qwen improvement in parallel

Before I change the backend architecture, I need these three launch decisions:

- Which Google Cloud project will hold the paid Gemini billing/API key?
- Do you already have a Google Play Console developer account under CLSL?
- For farmers, should sign-in be mobile OTP, Google sign-in, or both?

Once you confirm these, I will create the single CLSL AI Public Launch Master Plan and begin restructuring the production system around it.

10:36 AM

for sales officer only 4 photos compsostry , for other people it will be they can upload any number of imatge

i will create a new CLSL_AI project from my accoubnt and will use gemini 3.5 flash
 no we dont have Google Play Console developer account under CLSL
 sign-in be mobile OTP, Google sign-in, or both
 also sir want a normal login for all people and farmers and for our company dealer, like they have some qr code they give the to the farmer to famrer and farmer login thriugh it so we go the info of that this farmer is associate with this dealer , our main reason begeind all this is to collect the farmer data or after downloding there will be refer code scan to this , we want the data of farmers , so also dealer will tell downlaing and login thriugh this app give you some rs disciunt coupun inside it and that farmers can use and get dsicunt form the farmers and dealer give that much discunt to famrer, our mmain goal is to make the famers to touch with us directly as we have a huge dealers betwwen us , we are not drierclty serving to famrer, so thats why this app is insitate , to cnnect and collect famrer data

10:51 AM

Worked for 20s

This is the right direction. CLSL AI should become a direct farmer-connect platform, while still respecting the dealer network.

I have updated the launch design accordingly:

User type	Photos	Login	Purpose
-----------	--------	-------	---------

Sales Officer	Minimum 4	Personal CLSL field link	Structured field data and model-quality images
Farmer / public user	1–5 photos	Mobile number + OTP	Crop support, history and direct CLSL relationship
Dealer	Company login	Dealer dashboard	QR/referral codes, farmer attribution and coupons
CLSL admin	Secure company login	Admin portal	Catalogue, dealer controls, farmer data, campaigns and analytics

For public farmers, crop selection should be optional: they can select the crop, or choose “Other” and let the AI identify it. Sales Officers will still have the mandatory crop selection and four-photo flow.

The correct referral journey is:

Dealer QR / referral code



Farmer installs CLSL AI



Mobile OTP login + consent



Referral is saved permanently to farmer profile



Farmer uses crop inspection and CLSL AI



Eligible coupon appears in the app



Dealer validates coupon during purchase



CLSL sees farmer, dealer, location, crop and engagement data

The QR code must contain a signed referral token, not simply a visible dealer ID. This prevents farmers from changing or guessing dealer codes.

For login, I recommend:

- Mobile number + OTP as the main farmer sign-in.
- Optional Google sign-in later for convenience.
- Email/password or Microsoft login for CLSL employees, dealers and administrators.
- No login before a farmer can explore basic app information; login becomes compulsory before inspection, saved history, coupons or personal recommendations.

Your paid primary diagnosis model should be [gemini-3.5-flash](#). It supports image input and structured outputs, and Google lists it as a stable model. [Gemini 3.5 Flash documentation](#)

The production model roles will be:

Farmer crop inspection → Paid Gemini 3.5 Flash

Farmer/common chatbot → Paid Gemini 3.5 Flash

CLSL product selection → Deterministic CLSL catalogue engine

Qwen → Shadow evaluation and future fine-tuning only

The next public-launch blueprint will add these data areas:

- Farmer accounts, OTP verification and consent
- Dealer accounts, territories, QR codes and referral campaigns
- Referral attribution history
- Coupons, eligibility, redemption and abuse checks
- Farmer inspection history and feedback
- Gemini usage, latency and cost tracking
- Dealer/farmer analytics in the admin portal
- Public privacy, terms, data-deletion and support flows

You do not yet need a Play Console account to build the Android app, but CLSL will need one before releasing publicly on Play Store. We will create the Android app as a separate React Native/Expo client using the same secure backend as the web app.

The immediate next work should be the Public Launch Master Plan, followed by public authentication and the dealer QR/coupon system. I will keep the current CLSL AI interface work, but shift it toward this farmer-and-dealer launch model rather than only the Sales Officer pilot.

10:52 AM

so decide the plan in detail and we dont want to make a app using andorioed studio , we discussed that we will load the web app in the apk , like the amazon does , if you remembered we discussed thtis thing

10:55 AM

Yes, I remember: we will keep one CLSL AI web/PWA codebase and package it for Play Store, not build a separate React Native or Android Studio application. I'm confirming the safest Play Store packaging approach so the plan uses the web app directly without exposing API keys.

Searched the web for trusted web activity PWA Play Store Bubblewrap Digital Asset Links | [developer.android.com](#) ...

Context automatically compacted

You've hit your usage limit. Upgrade your plan or add credits to continue, or try again at Sep 19, 2026, 4:03 PM.

Today 12:46 PM

continue

12:46 PM

Worked for 48s

Correct—I remember. We will not build a separate Android application in Android Studio.

Final architecture

Android APK / Play Store app



Trusted Web Activity



<https://ai.croplifescience.com>



CLSL AI PWA



Secure backend API



PostgreSQL + S3 + Gemini + Qwen GPU

The APK will display the same CLSL AI web application. Any web update will immediately reach mobile users without rebuilding the Android app.

A Trusted Web Activity is designed specifically to open an owned PWA fullscreen inside an Android app. Bubblewrap can generate and build this package from the command line; Android Studio is not required for maintaining a native UI. Domain verification uses [assetlinks.json](#).
[Android Trusted Web Activity guide](#) · [Digital Asset Links](#)

Implementation plan

1. Complete the web/PWA first

- Final brand: **CLSL AI**
- Tagline: **Smart crop protection from Crop Life Science**
- CLSL logo and tomato doctor mascot
- Mobile-first responsive layout
- Camera capture and gallery upload
- Offline loading screen
- Install prompt and home-screen shortcut
- English, Hindi, Gujarati, Marathi, Bengali and Bhojpuri
- Light mode as default

- Admin routes excluded from the farmer APK

2. User accounts and dealer attribution

- Farmer login: mobile OTP
- Optional Google login
- Employee and admin login: email/password initially
- Dealer login and referral management
- Dealer QR link, for example:

<https://ai.croplifescience.com/join/secure-referral-token>

The QR will not expose raw database IDs. After the farmer verifies their mobile number, the system records the dealer association.

3. Farmer and dealer features

- Farmer profile with crop, city, state and territory
- Current weather and activity guidance
- Coupon wallet
- Dealer-linked discount coupon
- Farmer support through CLSL AI Mitra
- Dealer can see referral and coupon activity
- Dealers will not see unrelated farmer diagnoses or sensitive information

Coupon amount, expiry, eligible products and dealer redemption rules must be approved by CLSL management before launch.

4. AI inspection pipeline

Photo + selected crop



Image compression and secure S3 upload



Gemini 3.5 Flash diagnosis



Qwen shadow comparison



Consensus and confidence calculation



CLSL catalogue product engine



Category-wise product result



Weather, safety and sales-contact guidance

- Gemini 3.5 Flash will be the production-facing model.
- Qwen remains the private shadow/evaluation/fine-tuning model.
- The public user will never wait for Qwen.
- Product recommendations will come only from active approved CLSL catalogue records.
- Gemini will not invent products, doses or catalogue information.
- All image and result records will be retained for future Qwen improvement according to the company's retention policy.
- Sales Officers will provide exactly four structured field photos.
- Farmers and general users may upload a flexible number of photos.

Gemini API keys will remain only on the server, never inside the browser or APK.

5. Android packaging

Proposed package:

com.croplifescience.clslai

Required later:

- CLSL-owned Google Play Console organization account
- Company-owned signing key and recovery storage
- 192px and 512px app icons
- Splash screen and app metadata
- [/.well-known/assetlinks.json](#)
- Release AAB for Google Play
- APK only for internal testing or direct installation

The APK will be a thin secure shell. The actual features, API calls, AI logic and catalogue remain in the web application.

6. Launch sequence

1. Finish CLSL AI web/PWA.
2. Complete Gemini production configuration and billing.
3. Complete OTP, dealer QR and coupon workflows.
4. Verify S3, PostgreSQL, admin roles and audit logs.
5. Test on Android phones, desktop browsers and slow networks.
6. Generate internal APK.
7. Test with selected CLSL employees and sales officers.
8. Create the Play Store AAB.
9. Publish under the CLSL organization account.
10. Continue improving the web app without rebuilding the application shell.

Decisions still required from CLSL

- OTP provider: AWS SNS, MSG91, Twilio or another approved service
- Whether Google login is enabled at first launch
- Coupon value, expiry and eligible product rules
- Farmer image/data retention period
- Final Play Store organization account and owner
- Final Android app name and tagline

So the next technical step is: **finish and verify the CLSL AI PWA and secure production backend; only then package it as the Android APK.**